



UNIVERSITA' DEGLI STUDI DI TORINO

Dipartimento di Informatica

Corso di Laurea Magistrale in Informatica

Anno Accademico 2024/2025

Tecnologie del Linguaggio Naturale-Parte III Prof. L. Di Caro

Autore: Annalisa Sabatelli Matr. 866879

LAB-4

Realizzazione di una pipeline di *clustering* e *topic modeling*

1. Introduzione

L'obiettivo di questa attività di laboratorio è quello di realizzare una pipeline di clustering e topic modeling a partire da un dataset selezionato. In particolare è stato utilizzato un dataset costituito da circa 2,81M di articoli scientifici tratti da ArXiv, disponibile come risorsa open-access tramite la piattaforma Hugging Face alla URL:

<https://huggingface.co/datasets/librarian-bots/arxiv-metadata-snapshot>.

Ciascun articolo è caratterizzato dalle seguenti variabili principali:

- *Authors*: autori del documento;
- *Titles*: titolo dell'articolo;

- *Abstract*: riassunto del documento;
- *Categories*: classificazione dell'articolo secondo lo standard ArXiv.

All'interno di questo lavoro il numero di documenti considerato sarà limitato ai primi 40K per contenere i tempi di elaborazione.

2. Struttura del codice

La pipeline per il clustering testuale si articola in tre fasi principali:

1. Conversione dei documenti in vettori;
2. Riduzione della dimensionalità dei vettori di embeddings;
3. Clustering dei vettori in gruppi significativi.

Il codice è diviso in 4 file:

- *main.py*: indica il flusso di esecuzione dei metodi che compongono la pipeline e che sono definiti negli altri moduli del programma.
- *utility.py*: contiene metodi di utility di cui se ne riportano i principali:
 - *def data_dict_creation()*: Il metodo costruisce un dizionario Python a partire dal dataset di articoli scientifici selezionato. Per ogni elemento del dataset, viene estratto il titolo, l'abstract e le categorie. Successivamente sull'abstract viene applicato un processo di lemmatizzazione. Tutte queste informazioni vengono salvate in un dizionario indicizzato dall'indice dell'articolo che contiene sia il testo originale dell'abstract sia quello elaborato. Il ciclo si interrompe quando viene raggiunto il numero massimo di articoli specificato da NUM_ARTICLE.
 - *def extraction_lemmi_from_sentence(sentence, stop_words)*: metodo che prende in input una frase e il set di stop-words e la elabora restituendo una stringa costituita dai lemmi delle parole che la costituiscono. In particolare, l'elaborazione si articola nei seguenti passi:
 - tokenizzazione;
 - rappresentazione di tutte le parole in lower case;
 - rimozione delle stop-words;
 - rimozione della punteggiatura e/o numeri;
 - creazione di una stringa di lemmi separati da spazi.
- *text_clustering.py*: contiene i metodi che realizzano la pipeline per il task di text clustering di cui se ne riportano i principali:
 - *def embeddings_creation()*: prende in input la struttura dati dizionario degli articoli realizzata precedentemente e costruisce le rappresentazioni numeriche (embedding) degli abstract lemmatizzati. In particolare, estrae tutti i testi

preprocessati dal dizionario, quindi carica il modello pre-addestrato SentenceTransformer “*thenlper/gte-small*”, che è ottimizzato per catturare il significato semantico delle frasi. Applica il modello alla lista degli abstract e genera per ciascuno di essi un vettore denso che ne rappresenta il contenuto in uno spazio numerico. Il metodo restituisce sia l’array NumPy contenente tutti i vettori, sia il modello stesso.

- *def dim_reduce()*: il metodo riduce la dimensionalità degli embeddings usando la libreria UMAP (Uniform Manifold Approximation and Projection). In pratica, prende come input la matrice di embeddings generata in precedenza e la proietta in uno spazio con meno dimensioni, fissato da *n_components=5*. Questo permette di mantenere la struttura semantica dei dati, ma in uno spazio più compatto e più facile da gestire per compiti come la visualizzazione o il clustering. Il parametro *min_dist=0.0* indica che i punti possono essere compressi molto vicino, utile per catturare bene i cluster; *metric="cosine"* specifica che la similarità tra embeddings viene calcolata con la cosine similarity, adatta per dati testuali; infine, *random_state=42* garantisce riproducibilità dei risultati. Il metodo restituisce i nuovi embeddings ridotti (con forma (N, 5)) e l’oggetto *umap_model* allenato, che può essere riutilizzato.
- *def plot_umap_clusters()*: il metodo rappresenta graficamente i cluster ottenuti con HDBSCAN. Per rendere possibile la visualizzazione, la funzione applica prima una riduzione dimensionale con UMAP, trasformando gli embedding in uno spazio bidimensionale. Successivamente costruisce un DataFrame che contiene le coordinate 2D degli articoli. Per rendere più chiara la visualizzazione, la funzione separa gli outlier, cioè i punti che HDBSCAN non assegna a nessun cluster (label -1), dai punti che appartengono a cluster reali. Infine, utilizza Matplotlib per creare lo scatter plot: gli outlier vengono mostrati in grigio, mentre i punti dei cluster vengono colorati secondo la palette *tab20b* disponibile in Matplotlib.
- *topic_modeling.py*: contiene i metodi che realizzano la pipeline per il task di topic modeling di cui se ne riportano i principali:
 - *def BERTopic_modeling (model, umap_model, hdbscan_model, abstracts, embeddings)*: il metodo crea un modello di topic modeling basato su BERTopic, sfruttando gli embedding degli abstract e le informazioni ottenute dai modelli di riduzione dimensionale e clustering precedentemente calcolati. L’oggetto BERTopic utilizza i tre modelli per identificare automaticamente i topic presenti nei documenti. Il metodo *fit* applica il topic modeling sugli abstract, sfruttando gli embedding come rappresentazione dei contenuti. Infine, la funzione restituisce l’oggetto *topic_model*, che può essere utilizzato

successivamente per esplorare i singoli topic, estrarre le parole chiave, o assegnare nuovi documenti ai topic già identificati.

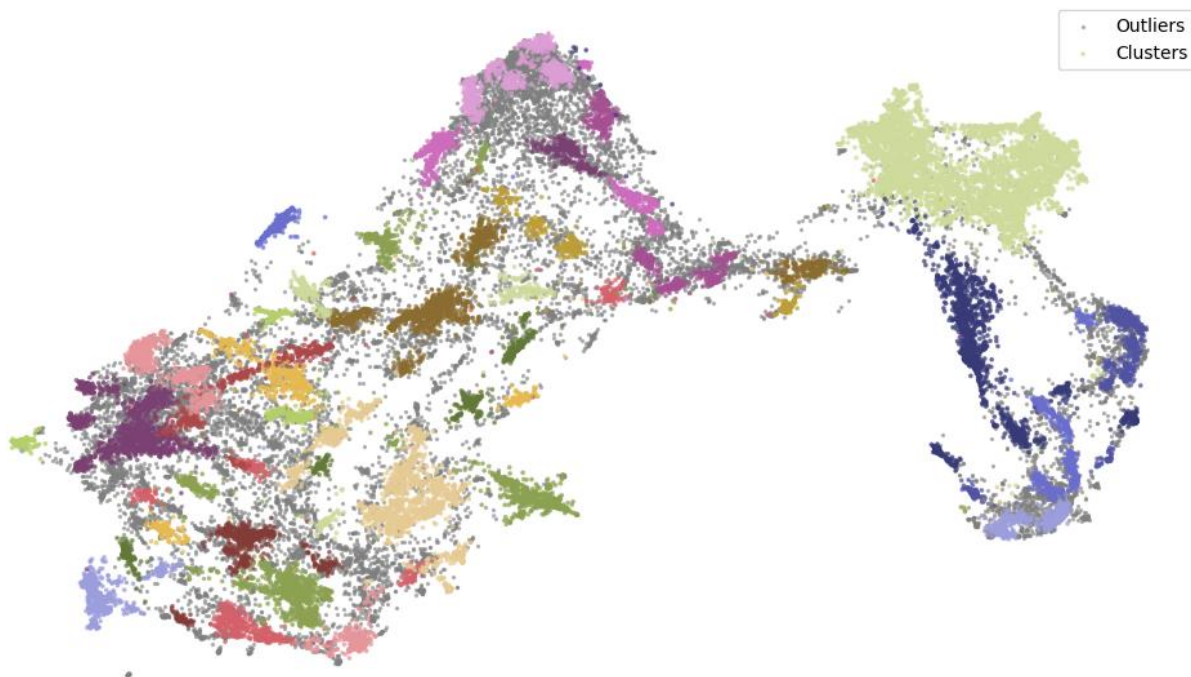
- *def topic_visualization (topic_model, abstracts, reduced_embeddings_2D):*
serve a rappresentare graficamente i risultati ottenuti da un modello BERTopic tramite barchart.

3. Risultati ottenuti

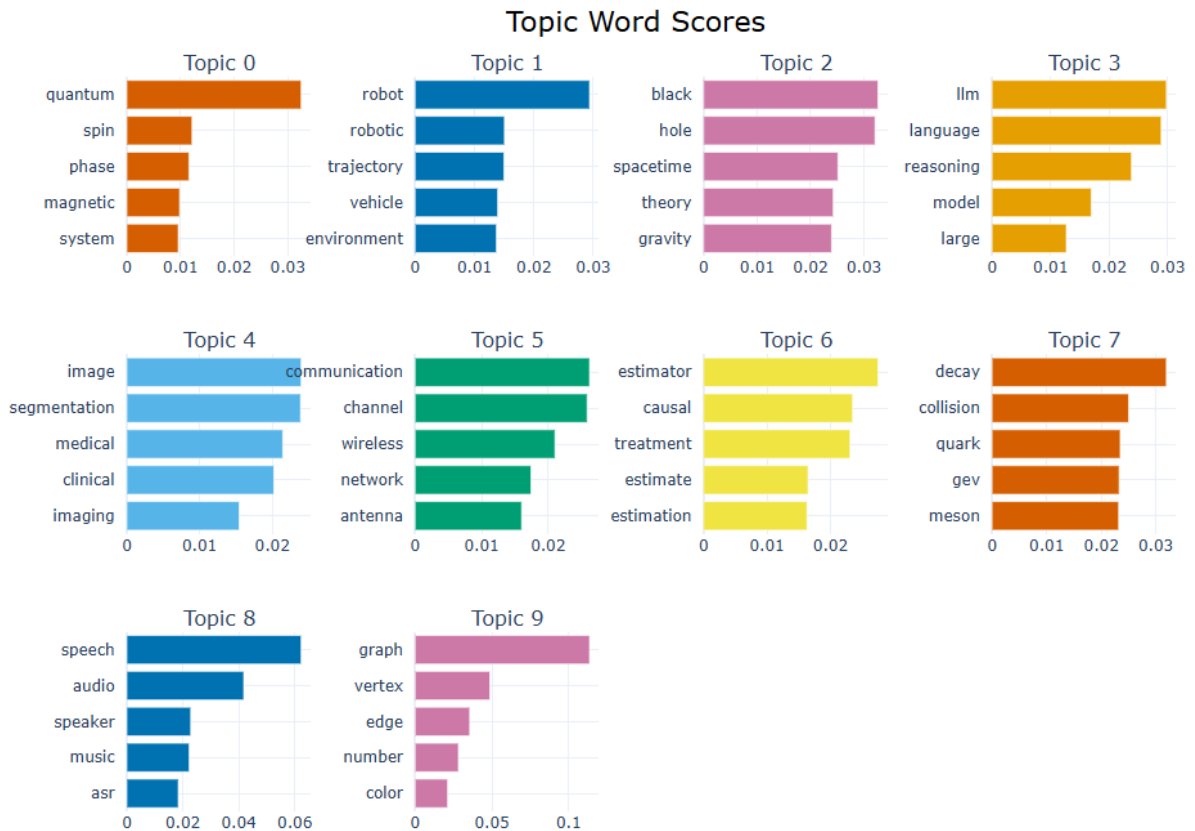
Si riportano i risultati ottenuti a valle dell'esecuzione del codice.

Task: Text clustering

Visualizzazione UMAP dei Cluster



Task: Topic Modeling



4. Conclusioni

L'attività svolta ha consentito di approfondire in maniera sistematica le potenzialità delle tecniche di analisi del linguaggio naturale applicate a un corpus testuale di dimensioni significative composto da 40.000 documenti. Attraverso l'impiego di rappresentazioni vettoriali, riduzione dimensionale e algoritmi di clustering non supervisionato, è stato possibile mettere in evidenza strutture semantiche latenti all'interno dei documenti, successivamente interpretate mediante il topic modeling con BERTopic. I risultati ottenuti hanno mostrato come tali metodologie siano in grado di organizzare e sintetizzare in maniera efficace l'informazione, pur evidenziando alcune lacune di interpretazione dei topic estratti. Non tutti i topic individuati dal modello presentano una coerenza semantica immediata. In alcuni casi, infatti, le parole chiave associate da BERTopic non formano un insieme facilmente interpretabile da un osservatore umano, rendendo più complesso attribuire un significato univoco al cluster. Un'ulteriore criticità riguarda la presenza di topic molto generici o ridondanti: può accadere che due gruppi distinti contengano termini molto simili, dando

più l'idea di una duplicazione piuttosto che di una reale differenza concettuale. Infine, è importante sottolineare che la qualità dei topic estratti dipende fortemente dal dataset di partenza: se gli abstract sono sbilanciati o eccessivamente tecnici, il modello può restituire raggruppamenti poco chiari.